

CorrelationWorkstation.c DSP Mathematics and Implementation Reference

August 11, 2026

Abstract

This document explains the digital signal processing calculations implemented by the correlation workstation. For every major calculation, it shows the relevant C code, expresses the same operation mathematically, defines the variables, and explains why the calculation is performed. The emphasis is the active correlation path: complex16 normalization, short-window power detection, adaptive noise-floor tracking, hysteretic burst extraction, 99-percent occupied-bandwidth estimation, magnitude/frequency/phase signature construction, phase unwrapping and detrending, z-score normalization, bounded-lag normalized cross-correlation, occupied-bandwidth similarity, energy-based query-burst selection, and weighted result ranking.

Contents

1	Scope and Signal Model	3
1.1	Important DSP constants	3
2	Recording Metadata and Physical Units	4
2.1	Center frequency and sample-rate conversion	4
3	Transmission Detection by Windowed Power	4
3.1	Detection-window length	4
3.2	Complex16 normalization and mean window power	5
3.3	Robust initial baseline from the lower power distribution	6
3.4	dB thresholds converted to linear power ratios	6
3.5	Adaptive exponential baseline	6
3.6	Hysteresis, end-hold, pretrigger, and minimum duration	7
3.7	Fallback when no burst is detected	8
4	99-Percent Occupied Bandwidth Estimation	9
4.1	FFT size and block placement	9
4.2	Hann window and zero padding	9
4.3	Forward DFT, FFT shift, and accumulated spectral power	10
4.4	Cumulative 0.5-percent and 99.5-percent power limits	10
5	Transmission Signature Construction	11
5.1	Burst time and duration	11
5.2	Mapping arbitrary burst length to a fixed trend length	12
5.3	Magnitude, linear power, peak power, and average power	12
5.4	Wrapped phase	13

5.5	Instantaneous frequency from adjacent complex samples	13
5.6	Phase unwrapping	14
5.7	Per-bucket frequency and phase averages	14
5.8	Frequency centering and linear phase detrending	15
6	Z-Score Trend Normalization	16
6.1	Final signature assembly	16
7	Bounded-Lag Normalized Cross-Correlation	17
7.1	Lag range and full-vector energy checks	17
7.2	Overlap-only normalized correlation at each lag	18
7.3	Why the lag search helps	18
8	Occupied-Bandwidth Similarity	19
9	Query-Burst Energy Selection	19
10	Weighted Similarity Score	20
10.1	Normalized weighted combination	20
10.2	Per-file best-signature selection	20
11	Trend Visualization Mathematics	21
12	End-to-End Active DSP Path	22
13	Worked Numerical Examples	22
13.1	Detector at 2 MS/s	22
13.2	4096-point occupied-bandwidth resolution at 2 MS/s	23
13.3	Instantaneous frequency from a quarter-turn phase step	23
13.4	Default weighted similarity	23
14	Engineering Notes and Interpretation	23
15	Compact Formula Reference	24
16	Core DSP Source Locations	24
17	Core DSP Calculation Listings	25
17.1	Power-window burst detection	25
17.2	Occupied-bandwidth FFT calculation	26
17.3	Magnitude, instantaneous-frequency, and unwrapped-phase calculation	27
17.4	Trend centering, detrending, and normalization	28
17.5	Bounded-lag normalized cross-correlation	28
17.6	Similarity components and composite score	29
17.7	Active candidate-ranking calculation	30

1 Scope and Signal Model

The correlation workstation reads signed interleaved complex16 recordings. One stored sample is interpreted as

$$x[n] = I[n] + jQ[n], \quad (1)$$

with normalized floating-point components

$$I_f[n] = \frac{I_{16}[n]}{32768}, \quad Q_f[n] = \frac{Q_{16}[n]}{32768}. \quad (2)$$

The corresponding instantaneous linear power is

$$p[n] = |x[n]|^2 = I_f[n]^2 + Q_f[n]^2. \quad (3)$$

The active signal-processing chain is

recording $\rightarrow$ short-window power $\rightarrow$ burst ranges,
 burst $\rightarrow$ {magnitude, instantaneous frequency, unwrapped phase, OBW},
 features $\rightarrow$ fixed-length normalized trends,
 query/candidate $\rightarrow$ bounded-lag normalized cross-correlation $\rightarrow$ weighted score.

The `TransmissionSignature` storage type and `CORRELATION_TREND_POINTS` array capacity are supplied by the project header. Within this source, the active trend length is user-configurable and defaults to 256 points.

1.1 Important DSP constants

Relevant code (CorrelationWorkstation.c lines 76–104):

```

76 #define CORRELATION_DETECTION_WINDOW_SEC 0.002
77 #define CORRELATION_BASELINE_TIME_CONSTANT_SEC 1.0
78 #define CORRELATION_START_DB 8.0
79 #define CORRELATION_END_DB 4.0
80 #define CORRELATION_END_HOLD_SEC 0.050
81 #define CORRELATION_PRETRIGGER_SEC 0.050
82 #define CORRELATION_MIN_BURST_SEC 0.004
83 #define CORRELATION_OBW_FFT_SIZE 4096
84 #define CORRELATION_OBW_MAX_BLOCKS 32
91 #define CORRELATION_MIN_TREND_POINTS 32
92 #define CORRELATION_MAX_TREND_POINTS CORRELATION_TREND_POINTS
93 #define CORRELATION_DEFAULT_TREND_POINTS 256
94 #define CORRELATION_MAX_LAG_PERCENT 0.125
95 #define CORRELATION_HIDDEN_YIELD_MS 4U
96 #define CORRELATION_HIDDEN_DETECTION_BATCH 8U
97 #define CORRELATION_HIDDEN_SIGNATURE_BATCH 2U
98 #define CORRELATION_HIDDEN_NCC_LAG_BATCH 8U
99 #define CORRELATION_HIDDEN_FILE_DELAY_MS 12U
101 /* Default score scalars; users can edit them in the Signature Engine bar. */
102 #define CORRELATION_DEFAULT_MAGNITUDE_WEIGHT 0.30
103 #define CORRELATION_DEFAULT_FREQUENCY_WEIGHT 0.60
104 #define CORRELATION_DEFAULT_BANDWIDTH_WEIGHT 0.10

```

The detector uses 2 ms power windows, a 1 s adaptive baseline time constant, an 8 dB start threshold, a 4 dB end threshold, 50 ms of end-hold, 50 ms of pretrigger, and a 4 ms minimum retained burst duration. Occupied bandwidth uses a 4096-point FFT and at most 32 analysis blocks. The default signature resolution is

$$P = 256 \text{ trend points}, \quad (4)$$

and the correlation lag search is bounded by

$$|k| \leq 0.125P. \quad (5)$$

The default score scalars are

$$w_m = 0.30, \quad w_f = 0.60, \quad w_B = 0.10. \quad (6)$$

2 Recording Metadata and Physical Units

2.1 Center frequency and sample-rate conversion

Relevant code (CorrelationWorkstation.c lines 372–415):

```

372 static void correlation_parse_recording_metadata(const char *name, double *sample_rate, double
    *center_frequency) {
373     /*
374      * Purpose: Parses sample rate and center frequency from a recording name
375      * Returns: No value
376      */
377     double parsed_sample_rate = Global_Correlation_Fallback_Record_Rate_Hz > 0
378         ? (double)Global_Correlation_Fallback_Record_Rate_Hz
379         : (double)Global_Correlation_Fallback_Sample_Rate_Hz;
380     capture = strstr(name, "_CAPTURE_");
381     if (capture) {
382         double mhz = 0.0;
383         if (sscanf(capture, "_CAPTURE_%lfMHz", &mhz) == 1 && mhz > 0.0) {
384             parsed_center = mhz * 1e6;
385         }
386     }
387     rate = strstr(name, "_SR_");
388     if (rate) {
389         double khz = 0.0;
390         if (sscanf(rate, "_SR_%lfk", &khz) == 1 && khz > 0.0) {
391             parsed_sample_rate = khz * 1000.0;
392         }
393     }
394 }

```

The recording filename stores center frequency in MHz and recording sample rate in kHz. The conversions are

$$f_c = f_{\text{MHz}} \times 10^6, \quad (7)$$

$$F_s = F_{\text{kHz}} \times 10^3. \quad (8)$$

All later time, frequency, and bandwidth calculations use these values in SI units. If the filename does not contain usable metadata, the source falls back to recording/sample-rate and center-frequency values supplied by the application.

3 Transmission Detection by Windowed Power

3.1 Detection-window length

Relevant code (CorrelationWorkstation.c lines 1033–1047):

```

1033 window_samples = (size_t)llround(sample_rate * CORRELATION_DETECTION_WINDOW_SEC);
1034 if (window_samples < 256) {
1035     window_samples = 256;
1036 }
1037 if (window_samples > 8192) {
1038     window_samples = 8192;
1039 }
1040 window_count = (iq_count + window_samples - 1) / window_samples;

```

The nominal number of samples in one power window is

$$N_w = \text{round}(F_s T_w), \quad (9)$$

where

$$T_w = 0.002 \text{ s}. \quad (10)$$

The implementation clamps the result to

$$256 \leq N_w \leq 8192. \quad (11)$$

The number of windows is a ceiling division,

$$N_{\text{win}} = \left\lceil \frac{N_{\text{IQ}}}{N_w} \right\rceil, \quad (12)$$

implemented by `(iq_count + window_samples - 1) / window_samples`.

3.2 Complex16 normalization and mean window power

Relevant code (CorrelationWorkstation.c lines 1074–1105):

```

1074     for (size_t window = 0; window < window_count; window++) {
1075         size_t remaining = iq_count - window * window_samples;
1076         size_t wanted = remaining < window_samples ? remaining : window_samples;
1077         size_t read_count;
1078         double sum = 0.0;
1079         if (Global_Correlation_Cancel) {
1080             free(samples);
1081             free(power);
1082             return 0;
1083         }
1084         read_count = fread(samples, sizeof(int16_t) * 2U, wanted, fp);
1085         if (read_count != wanted) {
1086             free(samples);
1087             free(power);
1088             return 0;
1089         }
1090         for (size_t i = 0; i < wanted; i++) {
1091             double real = (double)samples[2U * i] / 32768.0;
1092             double imag = (double)samples[2U * i + 1U] / 32768.0;
1093             sum += real * real + imag * imag;
1094         }
1095         power[window] = wanted > 0 ? sum / (double)wanted : 0.0;
1096     }

```

For every complex sample in a detection window,

$$x[n] = \frac{I_{16}[n]}{32768} + j \frac{Q_{16}[n]}{32768}. \quad (13)$$

Its power is

$$p[n] = |x[n]|^2 = I_f[n]^2 + Q_f[n]^2. \quad (14)$$

The detector reduces each short window to mean linear power,

$$P_w[r] = \frac{1}{N_r} \sum_{n=0}^{N_r-1} |x_r[n]|^2, \quad (15)$$

where N_r is the number of valid samples in window r . The last window may contain fewer than N_w samples.

3.3 Robust initial baseline from the lower power distribution

Relevant code (CorrelationWorkstation.c lines 1113–1133):

```

1113     initial_count = window_count < 500 ? window_count : 500;
1114     initial_values = (double *)malloc(initial_count * sizeof(double));
1116     if (!initial_values) {
1118         free(samples);
1119         free(power);
1120         return 0;
1122     }
1124     memcpy(initial_values, power, initial_count * sizeof(double));
1125     qsort(initial_values, initial_count, sizeof(double), correlation_double_compare);
1126     baseline = initial_values[(initial_count - 10) / 50];
1127     free(initial_values);
1129     if (baseline < 1e-15) {
1131         baseline = 1e-15;
1133     }

```

The detector copies at most the first 500 window-power values, sorts them, and chooses index

$$i_b = \left\lfloor \frac{N_0 - 1}{5} \right\rfloor. \quad (16)$$

Thus the initial baseline is approximately the 20th percentile,

$$B_0 \approx Q_{0.20}(P_w). \quad (17)$$

Using a low quantile rather than a mean reduces the influence of transmissions already present near the beginning of the recording. A floor of 10^{-15} prevents a zero or extremely small baseline from making the threshold numerically unusable.

3.4 dB thresholds converted to linear power ratios

Relevant code (CorrelationWorkstation.c lines 1017–1019):

```

1017     double alpha;
1018     double start_ratio = pow(10.0, CORRELATION_START_DB / 10.0);
1019     double end_ratio = pow(10.0, CORRELATION_END_DB / 10.0);

```

Because the detector compares *power*, a threshold of D dB corresponds to the linear ratio

$$R = 10^{D/10}. \quad (18)$$

Therefore the source uses

$$R_{\text{start}} = 10^{8/10} \approx 6.3096, \quad (19)$$

$$R_{\text{end}} = 10^{4/10} \approx 2.5119. \quad (20)$$

A burst begins when the short-window power rises above roughly 6.31 times the current baseline and is considered low enough to terminate when it falls below roughly 2.51 times the baseline.

3.5 Adaptive exponential baseline

Relevant code (CorrelationWorkstation.c lines 1135–1137 and 1145–1161):

```

1135     alpha = exp(-((double>window_samples / sample_rate) / CORRELATION_BASELINE_TIME_CONSTANT_SEC);
1136     low_windows_required = (int)ceil(CORRELATION_END_HOLD_SEC * sample_rate / (double>window_samples);
1137     pretrigger_windows = (int)ceil(CORRELATION_PRETRIGGER_SEC * sample_rate / (double>window_samples);
1145     for (size_t window = 0; window < window_count; window++) {
1146         double current = power[window];

```

```

1148     if (!active) {
1150         if (current > baseline * start_ratio) {
1152             active = 1;
1153             low_window_count = 0;
1154             start_window = window > (size_t)pretrigger_windows ? window - (size_t)pretrigger_windows : 0;
1156         }
1158     } else {
1160         baseline = alpha * baseline + (1.0 - alpha) * current;

```

The actual time represented by one detection window is

$$\Delta t = \frac{N_w}{F_s}. \quad (21)$$

For a baseline time constant $\tau = 1$ s, the exponential coefficient is

$$\alpha = e^{-\Delta t/\tau}. \quad (22)$$

When no transmission is active, the baseline update is

$$B_r = \alpha B_{r-1} + (1 - \alpha) P_w[r]. \quad (23)$$

This is a first-order exponential moving average whose response rate is defined in seconds rather than a fixed number of windows. The baseline is intentionally not updated from high-power windows after a start event, preventing the transmission itself from rapidly pulling the estimated noise floor upward.

3.6 Hysteresis, end-hold, pretrigger, and minimum duration

Relevant code (CorrelationWorkstation.c lines 1136–1205):

```

1136     low_windows_required = (int)ceil(CORRELATION_END_HOLD_SEC * sample_rate / (double>window_samples);
1137     pretrigger_windows = (int)ceil(CORRELATION_PRETRIGGER_SEC * sample_rate / (double>window_samples);
1148     if (!active) {
1150         if (current > baseline * start_ratio) {
1152             active = 1;
1153             low_window_count = 0;
1154             start_window = window > (size_t)pretrigger_windows ? window - (size_t)pretrigger_windows : 0;
1156         }
1158     } else {
1160         baseline = alpha * baseline + (1.0 - alpha) * current;
1162     }
1164 }
1166 else {
1168     if (current < baseline * end_ratio) {
1170         low_window_count++;
1172     }
1174     else {
1176         low_window_count = 0;
1178     }
1180     if (low_window_count >= low_windows_required) {
1182         size_t end_window = window + 1U - (size_t)low_window_count;
1183         size_t start_sample = start_window * window_samples;
1184         size_t end_sample = end_window * window_samples;
1186         if (end_sample > iq_count) {
1188             end_sample = iq_count;
1190         }
1192         if (end_sample > start_sample &&
1193             (double)(end_sample - start_sample) / sample_rate >= CORRELATION_MIN_BURST_SEC &&
1194             range_count < CORRELATION_MAX_SIGNATURES_PER_FILE) {
1196             ranges[range_count].start_sample = start_sample;
1197             ranges[range_count].end_sample = end_sample;
1198             range_count++;
1200         }
1202         active = 0;
1203         low_window_count = 0;
1204         baseline = alpha * baseline + (1.0 - alpha) * current;

```

The number of consecutive low-power windows required to end a burst is

$$N_{\text{hold}} = \left\lceil \frac{T_{\text{hold}} F_s}{N_w} \right\rceil, \quad (24)$$

and the pretrigger amount is

$$N_{\text{pre}} = \left\lceil \frac{T_{\text{pre}} F_s}{N_w} \right\rceil. \quad (25)$$

The start condition is

$$P_w[r] > B_r R_{\text{start}}. \quad (26)$$

When this occurs, the retained burst begins up to N_{pre} windows earlier. During an active burst, the end counter increments only when

$$P_w[r] < B_r R_{\text{end}}. \quad (27)$$

Any window that rises back above the end threshold resets that counter. This two-threshold state machine is hysteresis: the signal must cross a high threshold to start but only a lower threshold to remain active.

After the end hold is satisfied, the sample indices are

$$n_{\text{start}} = r_{\text{start}} N_w, \quad (28)$$

$$n_{\text{end}} = r_{\text{end}} N_w. \quad (29)$$

A range is retained only if

$$\frac{n_{\text{end}} - n_{\text{start}}}{F_s} \geq T_{\text{min}} = 0.004 \text{ s}. \quad (30)$$

3.7 Fallback when no burst is detected

Relevant code (CorrelationWorkstation.c lines 1211–1234):

```

1211     if (active && range_count < CORRELATION_MAX_SIGNATURES_PER_FILE) {
1213         size_t start_sample = start_window * window_samples;
1215         if (iq_count > start_sample && (double)(iq_count - start_sample) / sample_rate >=
CORRELATION_MIN_BURST_SEC) {
1217             ranges[range_count].start_sample = start_sample;
1218             ranges[range_count].end_sample = iq_count;
1219             range_count++;
1221         }
1223     }
1225     free(samples);
1226     free(power);
1228     if (range_count == 0 && iq_count > 0) {
1230         ranges[0].start_sample = 0;
1231         ranges[0].end_sample = iq_count;
1232         range_count = 1;
1234     }

```

If the recording ends while a burst is active, the final range extends to the final IQ sample, subject to the same minimum-duration requirement. If no range survives detection at all, the code deliberately falls back to

$$[n_{\text{start}}, n_{\text{end}}) = [0, N_{\text{IQ}}), \quad (31)$$

so the complete recording can still produce a signature rather than causing the file to be discarded solely because the detector did not find a threshold crossing.

4 99-Percent Occupied Bandwidth Estimation

4.1 FFT size and block placement

Relevant code (CorrelationWorkstation.c lines 1262–1306):

```

1262     segment_samples = end_sample - start_sample;
1263     input = (fftw_complex *)fftw_malloc(sizeof(fftw_complex) * CORRELATION_OBW_FFT_SIZE);
1264     output = (fftw_complex *)fftw_malloc(sizeof(fftw_complex) * CORRELATION_OBW_FFT_SIZE);
1265     spectrum = (double *)calloc(CORRELATION_OBW_FFT_SIZE, sizeof(double));
1266     samples = (int16_t *)malloc(CORRELATION_OBW_FFT_SIZE * 2U * sizeof(int16_t));
1267     if (!input || !output || !spectrum || !samples) {
1268         goto cleanup;
1269     }
1270     plan = fftw_plan_dft_1d(CORRELATION_OBW_FFT_SIZE, input, output, FFTW_FORWARD, FFTW_ESTIMATE);
1271     if (!plan) {
1272         goto cleanup;
1273     }
1274     if (segment_samples <= CORRELATION_OBW_FFT_SIZE) {
1275         block_step = 1;
1276     }
1277     else {
1278         block_step = (segment_samples - CORRELATION_OBW_FFT_SIZE) /
1279                     (size_t)(CORRELATION_OBW_MAX_BLOCKS > 1 ? CORRELATION_OBW_MAX_BLOCKS - 1 : 1);
1280         if (block_step < 1) {
1281             block_step = 1;
1282         }
1283     }
1284     for (size_t offset = 0;
1285          offset < segment_samples && block_count < CORRELATION_OBW_MAX_BLOCKS && !Global_Correlation_Cancel;
1286          offset += block_step) {
1287         size_t absolute = start_sample + offset;
1288         size_t available = end_sample - absolute;
1289         size_t wanted = available < CORRELATION_OBW_FFT_SIZE ? available : CORRELATION_OBW_FFT_SIZE;

```

Let the detected segment contain

$$N_s = n_{\text{end}} - n_{\text{start}} \quad (32)$$

samples and let the occupied-bandwidth FFT length be

$$N = 4096. \quad (33)$$

For long bursts the code chooses approximately uniform offsets across the segment using

$$S = \max\left(1, \left\lfloor \frac{N_s - N}{B_{\text{max}} - 1} \right\rfloor\right), \quad (34)$$

with $B_{\text{max}} = 32$. At most 32 FFT blocks are accumulated. For a segment no longer than one FFT, only one block is used.

4.2 Hann window and zero padding

Relevant code (CorrelationWorkstation.c lines 1321–1336):

```

1321     for (int i = 0; i < CORRELATION_OBW_FFT_SIZE; i++) {
1322         double window = 0.0;
1323         double real = 0.0;
1324         double imag = 0.0;
1325         if ((size_t)i < wanted) {
1326             window = 0.5 - 0.5 * cos((2.0 * M_PI * (double)i) / (double)(CORRELATION_OBW_FFT_SIZE - 1));
1327             real = (double)samples[2U * (size_t)i] / 32768.0;
1328             imag = (double)samples[2U * (size_t)i + 1U] / 32768.0;
1329         }
1330         input[i][0] = real * window;
1331         input[i][1] = imag * window;
1332     }

```

For valid sample index n , the Hann coefficient is

$$w[n] = \frac{1}{2} - \frac{1}{2} \cos\left(\frac{2\pi n}{N-1}\right). \quad (35)$$

The FFT input is

$$x_w[n] = w[n] (I_f[n] + jQ_f[n]). \quad (36)$$

If the selected block contains fewer than 4096 samples, all positions after the available data are explicitly zero because `real`, `imag`, and `window` remain zero. Thus the final transform is zero-padded to the fixed FFT length.

4.3 Forward DFT, FFT shift, and accumulated spectral power

Relevant code (CorrelationWorkstation.c lines 1338–1348):

```

1338     fftw_execute(plan);
1340     for (int i = 0; i < CORRELATION_OBW_FFT_SIZE; i++) {
1341         int shifted = (i + CORRELATION_OBW_FFT_SIZE / 2) % CORRELATION_OBW_FFT_SIZE;
1342         double real = output[i][0];
1343         double imag = output[i][1];
1344         spectrum[shifted] += real * real + imag * imag;
1345     }
1347     block_count++;
1348     correlation_worker_checkpoint(CORRELATION_HIDDEN_YIELD_MS);

```

FFTW computes the forward DFT

$$X[k] = \sum_{n=0}^{N-1} x_w[n] e^{-j2\pi kn/N}. \quad (37)$$

The power contribution of bin k is

$$|X[k]|^2 = \Re\{X[k]\}^2 + \Im\{X[k]\}^2. \quad (38)$$

The index

$$k_s = (k + N/2) \bmod N \quad (39)$$

performs an FFT shift so negative frequencies lie to the left of DC and positive frequencies lie to the right. Spectral powers are summed across the selected blocks,

$$S[k_s] \leftarrow S[k_s] + |X[k]|^2. \quad (40)$$

A division by the number of blocks is unnecessary for percentile boundaries because multiplying every bin by the same positive constant leaves the cumulative power fractions unchanged.

4.4 Cumulative 0.5-percent and 99.5-percent power limits

Relevant code (CorrelationWorkstation.c lines 1357–1395):

```

1357     if (block_count > 0) {
1359         double total = 0.0;
1360         double lower_target;
1361         double upper_target;
1362         double cumulative = 0.0;
1363         int lower_bin = 0;
1364         int upper_bin = CORRELATION_OBW_FFT_SIZE - 1;
1365         int lower_found = 0;
1367         for (int i = 0; i < CORRELATION_OBW_FFT_SIZE; i++) {
1368             total += spectrum[i];

```

```

1369     }
1371     lower_target = total * 0.005;
1372     upper_target = total * 0.995;
1374     if (total > DBL_MIN) {
1376         for (int i = 0; i < CORRELATION_OBW_FFT_SIZE; i++) {
1377             cumulative += spectrum[i];
1379             if (!lower_found && cumulative >= lower_target) {
1381                 lower_bin = i;
1382                 lower_found = 1;
1384             }
1386             if (cumulative >= upper_target) {
1388                 upper_bin = i;
1389                 break;
1391             }
1392         }
1394         bandwidth = ((double)(upper_bin - lower_bin + 1) / (double)CORRELATION_OBW_FFT_SIZE) *
sample_rate;

```

Total accumulated spectral power is

$$P_{\text{tot}} = \sum_{k=0}^{N-1} S[k]. \quad (41)$$

The lower and upper cumulative targets are

$$P_L = 0.005P_{\text{tot}}, \quad (42)$$

$$P_U = 0.995P_{\text{tot}}. \quad (43)$$

The code scans the shifted spectrum from low to high frequency and records the first bins whose cumulative sum reaches these targets. Therefore 0.5 percent of the power is excluded from each edge and the retained interval contains approximately 99 percent of total power.

The frequency spacing of one FFT bin is

$$\Delta f = \frac{F_s}{N}, \quad (44)$$

and the occupied bandwidth estimate is

$$B_{99} = (k_U - k_L + 1) \frac{F_s}{N}. \quad (45)$$

The explicit +1 counts both boundary bins.

5 Transmission Signature Construction

5.1 Burst time and duration

Relevant code (CorrelationWorkstation.c lines 1460–1465):

```

1460     memset(signature, 0, sizeof(*signature));
1461     segment_samples = range.end_sample - range.start_sample;
1462     signature->start_time = (double)range.start_sample / sample_rate;
1463     signature->duration = (double)segment_samples / sample_rate;
1464     signature->center_frequency = center_frequency;
1465     samples = (int16_t *)malloc(chunk_samples * 2U * sizeof(int16_t));

```

For a detected range $[n_0, n_1)$,

$$N_s = n_1 - n_0, \quad (46)$$

$$t_0 = \frac{n_0}{F_s}, \quad (47)$$

$$T = \frac{N_s}{F_s}. \quad (48)$$

The center frequency is stored as metadata, while the following feature calculations operate on the complex baseband samples within that range.

5.2 Mapping arbitrary burst length to a fixed trend length

Relevant code (CorrelationWorkstation.c lines 1500–1515):

```

1500     for (size_t i = 0; i < wanted; i++) {
1501         size_t relative_index = processed + i;
1502         int point = (int)((relative_index * (size_t)Global_Correlation_Trend_Points) / segment_samples);
1503         double real = (double)samples[2U * i] / 32768.0;
1504         double imag = (double)samples[2U * i + 1U] / 32768.0;
1505         double magnitude = hypot(real, imag);
1506         double power = real * real + imag * imag;
1508         if (point >= Global_Correlation_Trend_Points) {
1510             point = Global_Correlation_Trend_Points - 1;
1512         }
1514         magnitude_sum[point] += magnitude;
1515         magnitude_count[point]++;

```

Let P be the active number of trend points. A relative sample index $r \in [0, N_s - 1]$ maps to trend bucket

$$q(r) = \left\lfloor \frac{rP}{N_s} \right\rfloor, \quad (49)$$

with an explicit clamp to $P - 1$. This resamples every detected burst onto the same normalized time axis regardless of its original duration. Samples in each bucket are averaged rather than selecting only one sample.

5.3 Magnitude, linear power, peak power, and average power

Relevant code (CorrelationWorkstation.c lines 1503–1521 and 1576–1582):

```

1503         double real = (double)samples[2U * i] / 32768.0;
1504         double imag = (double)samples[2U * i + 1U] / 32768.0;
1505         double magnitude = hypot(real, imag);
1506         double power = real * real + imag * imag;
1508         if (point >= Global_Correlation_Trend_Points) {
1510             point = Global_Correlation_Trend_Points - 1;
1512         }
1514         magnitude_sum[point] += magnitude;
1515         magnitude_count[point]++;
1516         total_power += power;
1518         if (power > peak_power) {
1520             peak_power = power;
1576     for (int point = 0; point < Global_Correlation_Trend_Points; point++) {
1577         signature->magnitude_trend[point] =
1578             magnitude_count[point] > 0 ? (float)(magnitude_sum[point] / (double)magnitude_count[point]) :
0.0f;
1579         signature->frequency_trend[point] =
1580             frequency_count[point] > 0 ? (float)(frequency_sum[point] / (double)frequency_count[point]) :
0.0f;
1581         signature->phase_trend[point] =
1582             phase_count[point] > 0 ? (float)(phase_sum[point] / (double)phase_count[point]) : 0.0f;

```

Per sample,

$$m[n] = \sqrt{I_f[n]^2 + Q_f[n]^2}, \quad (50)$$

$$p[n] = I_f[n]^2 + Q_f[n]^2 = m[n]^2. \quad (51)$$

For trend bucket q , the magnitude feature is

$$M[q] = \frac{1}{N_q} \sum_{n \in q} m[n]. \quad (52)$$

The scalar peak and average powers are

$$P_{\text{peak}} = \max_n p[n], \quad (53)$$

$$P_{\text{avg}} = \frac{1}{N_s} \sum_{n=0}^{N_s-1} p[n]. \quad (54)$$

Magnitude trend shape is later normalized before comparison; average power remains useful for choosing which detected query burst represents the query recording.

5.4 Wrapped phase

Relevant code (CorrelationWorkstation.c lines 1524–1526):

```
1524     {
1525         double phase = atan2(imag, real);
```

The principal complex phase is

$$\phi[n] = \text{atan2}(Q_f[n], I_f[n]), \quad (55)$$

with output in the interval $(-\pi, \pi]$.

5.5 Instantaneous frequency from adjacent complex samples

Relevant code (CorrelationWorkstation.c lines 1527–1532):

```
1527         if (have_previous) {
1529             double product_real = real * previous_real + imag * previous_imag;
1530             double product_imag = imag * previous_real - real * previous_imag;
1531             double instantaneous_frequency = (sample_rate / (2.0 * M_PI)) * atan2(product_imag,
product_real);
1532             double phase_delta = phase - previous_phase;
```

For adjacent samples, the code forms the product

$$z[n] = x[n]x^*[n-1]. \quad (56)$$

Expanding this product gives

$$\Re\{z[n]\} = I_n I_{n-1} + Q_n Q_{n-1}, \quad (57)$$

$$\Im\{z[n]\} = Q_n I_{n-1} - I_n Q_{n-1}, \quad (58)$$

which exactly matches `product_real` and `product_imag`. Its angle is the wrapped phase increment,

$$\Delta\phi[n] = \arg(x[n]x^*[n-1]). \quad (59)$$

The instantaneous frequency estimate is

$$f_{\text{inst}}[n] = \frac{F_s}{2\pi} \Delta\phi[n]. \quad (60)$$

A positive sample-to-sample phase rotation therefore produces positive frequency and a negative rotation produces negative frequency.

5.6 Phase unwrapping

Relevant code (CorrelationWorkstation.c lines 1532–1556):

```

1532         double phase_delta = phase - previous_phase;
1533         while (phase_delta > M_PI) {
1534             phase_delta -= 2.0 * M_PI;
1535         }
1536         while (phase_delta < -M_PI) {
1537             phase_delta += 2.0 * M_PI;
1538         }
1539         unwrapped_phase += phase_delta;
1540         frequency_sum[point] += instantaneous_frequency;
1541         frequency_count[point]++;
1542     }
1543     else {
1544         unwrapped_phase = phase;
1545     }
1546     phase_sum[point] += unwrapped_phase;
1547     phase_count[point]++;
1548     previous_phase = phase;

```

Direct subtraction of two principal phases can jump by nearly 2π when the angle crosses the $\pm\pi$ branch cut. The source repeatedly adjusts the phase difference so

$$-\pi \leq \Delta\phi_{\text{unwrap}} \leq \pi. \quad (61)$$

The cumulative unwrapped phase is then

$$\Phi[n] = \Phi[n-1] + \Delta\phi_{\text{unwrap}}[n]. \quad (62)$$

The phase signature stores bucket averages of $\Phi[n]$, while the frequency signature stores bucket averages of $f_{\text{inst}}[n]$.

5.7 Per-bucket frequency and phase averages

Relevant code (CorrelationWorkstation.c lines 1542–1555 and 1576–1596):

```

1542         unwrapped_phase += phase_delta;
1543         frequency_sum[point] += instantaneous_frequency;
1544         frequency_count[point]++;
1545     }
1546     else {
1547         unwrapped_phase = phase;
1548     }
1549     phase_sum[point] += unwrapped_phase;
1550     phase_count[point]++;
1551     for (int point = 0; point < Global_Correlation_Trend_Points; point++) {
1552         signature->magnitude_trend[point] =
1553             magnitude_count[point] > 0 ? (float)(magnitude_sum[point] / (double)magnitude_count[point]) :
1554             0.0f;
1555         signature->frequency_trend[point] =
1556             frequency_count[point] > 0 ? (float)(frequency_sum[point] / (double)frequency_count[point]) :
1557             0.0f;
1558         signature->phase_trend[point] =
1559             phase_count[point] > 0 ? (float)(phase_sum[point] / (double)phase_count[point]) : 0.0f;
1560         if (frequency_count[point] > 0) {
1561             frequency_mean += signature->frequency_trend[point];
1562             frequency_points++;
1563         }
1564     }
1565     if (frequency_points > 0) {
1566         frequency_mean /= (double)frequency_points;
1567     }

```

For each trend bucket q ,

$$F[q] = \frac{1}{N_{f,q}} \sum_{n \in q} f_{\text{inst}}[n], \quad (63)$$

$$\Phi[q] = \frac{1}{N_{\phi,q}} \sum_{n \in q} \Phi[n]. \quad (64)$$

The mean of the populated frequency buckets is

$$\bar{F} = \frac{1}{P_f} \sum_{q \in \mathcal{P}_f} F[q], \quad (65)$$

where $\mathcal{P}_f$ is the set of buckets containing at least one instantaneous-frequency observation.

5.8 Frequency centering and linear phase detrending

Relevant code (CorrelationWorkstation.c lines 1598–1609):

```

1598     for (int point = 0; point < Global_Correlation_Trend_Points; point++) {
1599         double phase_position =
1600             Global_Correlation_Trend_Points > 1 ? (double)point / (double)(Global_Correlation_Trend_Points -
1601             1) : 0.0;
1602         double phase_line =
1603             ((double)signature->phase_trend[0] +
1604             ((double)signature->phase_trend[Global_Correlation_Trend_Points - 1] -
1605             (double)signature->phase_trend[0]) *
1606             phase_position;
1607         signature->frequency_trend[point] = (float)((double)signature->frequency_trend[point] -
1608             frequency_mean);
1609         /* Remove constant carrier rotation so the graph shows phase-shape changes. */
1610         signature->phase_trend[point] = (float)((double)signature->phase_trend[point] - phase_line);
1611     }

```

The frequency trend is centered by

$$F_c[q] = F[q] - \bar{F}. \quad (66)$$

This removes a constant average frequency offset so comparison focuses on changes in frequency over the burst rather than absolute residual carrier offset.

For the phase trend, define normalized trend position

$$u_q = \frac{q}{P - 1}. \quad (67)$$

The straight line connecting the first and final phase-trend values is

$$L[q] = \Phi[0] + (\Phi[P - 1] - \Phi[0]) u_q. \quad (68)$$

The stored phase-shape trend becomes

$$\Phi_c[q] = \Phi[q] - L[q]. \quad (69)$$

A constant frequency offset creates a linear phase ramp; subtracting this endpoint line suppresses that overall ramp and preserves deviations from linear phase evolution.

6 Z-Score Trend Normalization

Relevant code (CorrelationWorkstation.c lines 961–1000):

```

961 static void correlation_normalize_trend(float values[CORRELATION_TREND_POINTS], int point_count) {
962     /*
963         Purpose: Z-score normalizes the active portion of a trend
964         Returns: No value
965     */
966     double mean = 0.0;
967     double variance = 0.0;
968     if (!values || point_count < 1 || point_count > CORRELATION_TREND_POINTS) {
969         return;
970     }
971     for (int i = 0; i < point_count; i++) {
972         mean += values[i];
973     }
974     mean /= (double)point_count;
975     for (int i = 0; i < point_count; i++) {
976         double centered = (double)values[i] - mean;
977         variance += centered * centered;
978     }
979     variance /= (double)point_count;
980     if (variance <= 1e-18) {
981         memset(values, 0, sizeof(float) * (size_t)point_count);
982         return;
983     }
984     double standard_deviation = sqrt(variance);
985     for (int i = 0; i < point_count; i++) {
986         values[i] = (float)(((double)values[i] - mean) / standard_deviation);
987     }
988 }

```

For an active trend $v[0], \dots, v[P-1]$, the mean is

$$\mu = \frac{1}{P} \sum_{i=0}^{P-1} v[i]. \quad (70)$$

The implementation uses population variance,

$$\sigma^2 = \frac{1}{P} \sum_{i=0}^{P-1} (v[i] - \mu)^2, \quad (71)$$

and standard deviation

$$\sigma = \sqrt{\sigma^2}. \quad (72)$$

When variance is sufficiently large, each value becomes

$$z[i] = \frac{v[i] - \mu}{\sigma}. \quad (73)$$

Thus the trend has approximately zero mean and unit variance. This makes magnitude and frequency shape comparison substantially less sensitive to absolute amplitude or absolute frequency scale. If variance is at or below 10^{-18} , the active trend is replaced by zeros because there is effectively no shape variation to normalize.

6.1 Final signature assembly

Relevant code (CorrelationWorkstation.c lines 1611–1617):

```

1611 correlation_normalize_trend(signature->magnitude_trend, Global_Correlation_Trend_Points);
1612 correlation_normalize_trend(signature->frequency_trend, Global_Correlation_Trend_Points);
1613 correlation_normalize_trend(signature->phase_trend, Global_Correlation_Trend_Points);

```



```

1614 signature->peak_power = peak_power;
1615 signature->average_power = segment_samples > 0 ? total_power / (double)segment_samples : 0.0;
1616 signature->occupied_bandwidth =
1617     correlation_estimate_occupied_bandwidth(fp, range.start_sample, range.end_sample, sample_rate);

```

The source independently z-score normalizes the magnitude, frequency, and detrended phase arrays, then stores peak power, average power, and the 99-percent occupied-bandwidth estimate. Conceptually one transmission signature is

$$\mathcal{S} = \{\mathbf{z}_M, \mathbf{z}_F, \mathbf{z}_\Phi, P_{\text{peak}}, P_{\text{avg}}, B_{99}, t_0, T, f_c\}. \quad (74)$$

Not every stored feature currently participates in the ranking score. The active similarity score uses magnitude trend, frequency trend, and occupied bandwidth; phase trend and several scalar metadata fields remain available for display and future scoring logic.

7 Bounded-Lag Normalized Cross-Correlation

7.1 Lag range and full-vector energy checks

Relevant code (CorrelationWorkstation.c lines 1687–1728):

```

1687 double best = 0.0;
1688 double full_left_energy = 0.0;
1689 double full_right_energy = 0.0;
1690 int point_count = Global_Correlation_Trend_Points;
1691 int maximum_lag;
1693 if (point_count < CORRELATION_MIN_TREND_POINTS) {
1695     point_count = CORRELATION_MIN_TREND_POINTS;
1697 }
1699 if (point_count > CORRELATION_MAX_TREND_POINTS) {
1701     point_count = CORRELATION_MAX_TREND_POINTS;
1703 }
1705 maximum_lag = (int)((double)point_count * CORRELATION_MAX_LAG_PERCENT);
1707 if (maximum_lag < 1) {
1709     maximum_lag = 1;
1711 }
1713 for (int i = 0; i < point_count; i++) {
1714     full_left_energy += (double)left[i] * (double)left[i];
1715     full_right_energy += (double)right[i] * (double)right[i];
1716 }
1718 if (full_left_energy <= DBL_MIN && full_right_energy <= DBL_MIN) {
1720     return 1.0;
1722 }
1724 if (full_left_energy <= DBL_MIN || full_right_energy <= DBL_MIN) {
1726     return 0.0;
1728 }

```

Let P be the active trend length. The maximum lag is

$$K = \max(1, \lfloor 0.125P \rfloor). \quad (75)$$

The search considers

$$k \in \{-K, -K + 1, \dots, K\}. \quad (76)$$

Before the lag loop, full-vector energies are

$$E_x = \sum_i x[i]^2, \quad (77)$$

$$E_y = \sum_i y[i]^2. \quad (78)$$

If both are effectively zero, the trends are treated as identical and return 1. If only one is effectively zero, similarity returns 0.

7.2 Overlap-only normalized correlation at each lag

Relevant code (CorrelationWorkstation.c lines 1730–1761):

```

1730     for (int lag = -maximum_lag; lag <= maximum_lag; lag++) {
1731         double numerator = 0.0;
1732         double left_energy = 0.0;
1733         double right_energy = 0.0;
1734         int count = 0;
1735         for (int i = 0; i < point_count; i++) {
1736             int j = i + lag;
1737             if (j < 0 || j >= point_count) {
1738                 continue;
1739             }
1740             numerator += (double)left[i] * (double)right[j];
1741             left_energy += (double)left[i] * (double)left[i];
1742             right_energy += (double)right[j] * (double)right[j];
1743             count++;
1744         }
1745         if (count >= point_count / 2 && left_energy > DBL_MIN && right_energy > DBL_MIN) {
1746             double score = numerator / sqrt(left_energy * right_energy);
1747             if (score > best) {
1748                 best = score;
1749             }
1750         }
1751     }

```

For a candidate lag k , valid pairs satisfy

$$j = i + k, \quad 0 \leq j < P. \quad (79)$$

Over only these overlapping indices, the code accumulates

$$C_k = \sum_i x[i]y[i + k], \quad (80)$$

$$E_{x,k} = \sum_i x[i]^2, \quad (81)$$

$$E_{y,k} = \sum_i y[i + k]^2. \quad (82)$$

The normalized correlation is

$$\rho[k] = \frac{C_k}{\sqrt{E_{x,k}E_{y,k}}}. \quad (83)$$

This is cosine-normalized cross-correlation over the overlap. Because the complete trends were already z-score normalized, it behaves similarly to shape correlation while permitting modest timing displacement.

The source accepts a lag score only if at least half the trend overlaps, then retains

$$\rho_{\max} = \max_k \rho[k]. \quad (84)$$

The initial best value is zero and the final value is clamped to $[0, 1]$, so negative correlation is intentionally not converted into positive similarity.

7.3 Why the lag search helps

Without lag search, two otherwise identical burst shapes displaced by several trend buckets can score poorly. Maximizing normalized correlation over $\pm 12.5\%$ of the trend length permits small timing offsets while still limiting the amount of alignment freedom. For the default $P = 256$,

$$K = \lfloor 256 \times 0.125 \rfloor = 32 \quad (85)$$

trend points.

8 Occupied-Bandwidth Similarity

Relevant code (CorrelationWorkstation.c lines 1773–1789):

```

1773 static double correlation_bandwidth_similarity(double left, double right) {
1774     /*
1775         Purpose: Compares occupied bandwidth values
1776         Returns: Similarity from zero to one
1777     */
1778     double maximum = left > right ? left : right;
1779     double minimum = left < right ? left : right;
1780     if (maximum <= 1.0) {
1781         return maximum == minimum ? 1.0 : 0.0;
1782     }
1783     return correlation_clamp_double(minimum / maximum, 0.0, 1.0);
1784 }

```

For practical bandwidth values greater than 1 Hz, the score is

$$S_B = \frac{\min(B_q, B_c)}{\max(B_q, B_c)}. \quad (86)$$

Therefore identical bandwidths score 1, a candidate with half or twice the query bandwidth scores 0.5, and increasingly different bandwidths approach zero. This ratio is symmetric:

$$S_B(B_q, B_c) = S_B(B_c, B_q). \quad (87)$$

The special branch for values at or below 1 returns 1 only when the two values are exactly equal; otherwise it returns 0.

9 Query-Burst Energy Selection

Relevant code (CorrelationWorkstation.c lines 1791–1804 and 1990–2002):

```

1791 static double correlation_signature_energy(const TransmissionSignature *signature) {
1792     /*
1793         Purpose: Ranks candidate query bursts by approximate captured energy
1794         Returns: Energy score
1795     */
1796     if (!signature) {
1797         return 0.0;
1798     }
1799     return signature->average_power * signature->duration;
1800 }
1801
1802 for (uint32_t i = 0; i < Global_Correlation_Cache[query_cache_index].signature_count; i++) {
1803     double energy =
1804     correlation_signature_energy(&Global_Correlation_Cache[query_cache_index].signatures[i]);
1805     if (energy > best_query_energy) {
1806         best_query_energy = energy;
1807         best_query_signature = (int)i;
1808     }
1809 }
1810
1811 Global_Correlation_Query_Signature =
1812 Global_Correlation_Cache[query_cache_index].signatures[best_query_signature];
1813
1814 Global_Correlation_Query_Signature_Valid = 1;

```

A recording may contain multiple detected transmissions. The query recording is represented by the retained transmission with the largest

$$E_{\text{proxy}} = P_{\text{avg}} T. \quad (88)$$

Since

$$P_{\text{avg}} = \frac{1}{N_s} \sum_n p[n], \quad T = \frac{N_s}{F_s}, \quad (89)$$

we obtain

$$E_{\text{proxy}} = \frac{1}{F_s} \sum_n p[n]. \quad (90)$$

For bursts from the same recording/sample rate, maximizing `average_power * duration` is therefore equivalent to maximizing summed sample power up to the common factor $1/F_s$. This favors a strong, sustained burst rather than simply the highest single-sample peak.

10 Weighted Similarity Score

10.1 Normalized weighted combination

Relevant code (CorrelationWorkstation.c lines 1891–1910):

```

1891 static double correlation_calculate_score(double magnitude_score, double frequency_score, double
      bandwidth_score) {
1892     /*
1893      * Purpose: Combines component similarities using user-entered score scalars
1894      * Returns: Normalized score from zero to one
1895      */
1896     double weight_sum =
1897         Global_Correlation_Magnitude_Weight + Global_Correlation_Frequency_Weight +
1898         Global_Correlation_Bandwidth_Weight;
1900     if (weight_sum <= DBL_MIN) {
1901         return 0.0;
1902     }
1903     return correlation_clamp_double((Global_Correlation_Magnitude_Weight * magnitude_score +
1904                                     Global_Correlation_Frequency_Weight * frequency_score +
1905                                     Global_Correlation_Bandwidth_Weight * bandwidth_score) /
1906                                     weight_sum,
1907                                     0.0, 1.0);
1910

```

Let

S_M = maximum normalized magnitude-trend correlation,

S_F = maximum normalized frequency-trend correlation,

S_B = occupied-bandwidth similarity.

The final score is

$$S = \frac{w_M S_M + w_F S_F + w_B S_B}{w_M + w_F + w_B}, \quad (91)$$

then clamped to $[0, 1]$. Dividing by the weight sum means the user-entered values act as relative scalars; they do not have to add to 1.

With the defaults,

$$S = 0.30S_M + 0.60S_F + 0.10S_B, \quad (92)$$

because the default weights already sum to 1. Frequency-shape similarity therefore contributes the largest share of the default ranking.

10.2 Per-file best-signature selection

Relevant code (CorrelationWorkstation.c lines 2051–2074):

```

2051     for (uint32_t signature_index = 0; signature_index <
      Global_Correlation_Cache[cache_index].signature_count;
2052          signature_index++) {
2053         TransmissionSignature *stored =
      &Global_Correlation_Cache[cache_index].signatures[signature_index];
2054         double magnitude_score = correlation_maximum_normalized_cross_correlation(
2055             Global_Correlation_Query_Signature.magnitude_trend, stored->magnitude_trend);

```

```

2056     double frequency_score = correlation_maximum_normalized_cross_correlation(
2057         Global_Correlation_Query_Signature.frequency_trend, stored->frequency_trend);
2058     double bandwidth_score = correlation_bandwidth_similarity(
2059         Global_Correlation_Query_Signature.occupied_bandwidth, stored->occupied_bandwidth);
2060     double score = correlation_calculate_score(magnitude_score, frequency_score, bandwidth_score);
2062     correlation_worker_checkpoint(CORRELATION_HIDDEN_YIELD_MS);
2064     if (!have_result || score > best_result.score) {
2066         best_result.signature_index = (int)signature_index;
2067         best_result.score = score;
2068         best_result.magnitude_score = magnitude_score;
2069         best_result.frequency_score = frequency_score;
2070         best_result.bandwidth_score = bandwidth_score;
2071         best_result.signature = *stored;
2072         have_result = 1;
2074     }

```

Every retained candidate transmission in a recording is compared with the selected query signature. For candidate signature s ,

$$S_M^{(s)} = \text{NCC}_{\max}(\mathbf{z}_{M,q}, \mathbf{z}_{M,s}), \quad (93)$$

$$S_F^{(s)} = \text{NCC}_{\max}(\mathbf{z}_{F,q}, \mathbf{z}_{F,s}), \quad (94)$$

$$S_B^{(s)} = \frac{\min(B_q, B_s)}{\max(B_q, B_s)}, \quad (95)$$

$$S^{(s)} = \text{WeightedScore}(S_M^{(s)}, S_F^{(s)}, S_B^{(s)}). \quad (96)$$

Only the highest-scoring signature for that candidate recording becomes the file's result. Therefore one recording receives one ranked result even if it contains multiple detected bursts.

11 Trend Visualization Mathematics

Relevant code (CorrelationWorkstation.c lines 2727–2760):

```

2727     double maximum = 0.0;
2728     int point_count = Global_Correlation_Trend_Points;
2730     if (!renderer || !trend || rect.w < 2 || rect.h < 2 || point_count < 2) {
2732         return;
2734     }
2736     for (int i = 0; i < point_count; i++) {
2737         double absolute = fabs((double)trend[i]);
2739         if (absolute > maximum) {
2741             maximum = absolute;
2743         }
2744     }
2746     if (maximum < 1e-9) {
2748         maximum = 1.0;
2750     }
2752     SDL_SetRenderDrawColor(renderer, color.r, color.g, color.b, color.a);
2754     for (int i = 1; i < point_count; i++) {
2755         int x0 = rect.x + ((i - 1) * (rect.w - 1)) / (point_count - 1);
2756         int x1 = rect.x + (i * (rect.w - 1)) / (point_count - 1);
2757         int y0 = rect.y + rect.h / 2 - (int)((double)trend[i - 1] / maximum) * (double)(rect.h / 2 - 3));
2758         int y1 = rect.y + rect.h / 2 - (int)((double)trend[i] / maximum) * (double)(rect.h / 2 - 3));
2760         SDL_RenderDrawLine(renderer, x0, y0, x1, y1);

```

The graphs do not alter the cached trend values. For display only, the largest absolute plotted value is

$$A = \max_i |z[i]|, \quad (97)$$

with a fallback $A = 1$ for an essentially flat trend. Point i maps horizontally to

$$x_i = x_0 + \frac{i(W - 1)}{P - 1}, \quad (98)$$

and vertically to

$$y_i = y_0 + \frac{H}{2} - \frac{z[i]}{A} \left(\frac{H}{2} - 3 \right). \quad (99)$$

Thus zero lies at the vertical midpoint, positive values plot upward, negative values plot downward, and the largest absolute trend excursion nearly reaches the graph boundary.

12 End-to-End Active DSP Path

For one recording, the active extraction path can be written as

$$\{I_{16}, Q_{16}\} \rightarrow P_w[r] \rightarrow \text{hysteretic ranges} \rightarrow \mathcal{S}_1, \dots, \mathcal{S}_R. \quad (100)$$

For one retained burst,

$$\mathcal{S} = \{Z(\text{bucketavg } |x|), Z(\text{bucketavg } f_{\text{inst}} - \bar{f}), Z(\text{bucketavg } \Phi - \text{line}(\Phi)), B_{99}, P_{\text{avg}}, P_{\text{peak}}, T\}, \quad (101)$$

where $Z(\cdot)$ denotes z-score normalization.

For a candidate signature,

$$S = \frac{w_M \max_k \rho_M[k] + w_F \max_k \rho_F[k] + w_B S_B}{w_M + w_F + w_B}. \quad (102)$$

The query recording contributes its highest-energy retained burst; each candidate recording contributes its highest-scoring retained burst.

13 Worked Numerical Examples

13.1 Detector at 2 MS/s

For

$$F_s = 2,000,000 \text{ samples/s}, \quad (103)$$

the nominal 2 ms detector window contains

$$N_w = 2,000,000 \times 0.002 = 4000 \text{ samples}. \quad (104)$$

This lies inside the source clamp of 256 to 8192.

The baseline coefficient is

$$\alpha = e^{-(4000/2,000,000)/1} = e^{-0.002} \approx 0.998002. \quad (105)$$

The 50 ms end-hold and pretrigger each become

$$\left\lceil \frac{0.050 \times 2,000,000}{4000} \right\rceil = 25 \text{ windows}. \quad (106)$$

The minimum 4 ms burst corresponds to

$$0.004 \times 2,000,000 = 8000 \text{ samples}. \quad (107)$$

The start and end ratios are approximately 6.31 and 2.51, respectively.

13.2 4096-point occupied-bandwidth resolution at 2 MS/s

The OBW bin spacing is

$$\Delta f = \frac{2,000,000}{4096} = 488.28125 \text{ Hz.} \quad (108)$$

If the cumulative 0.5-percent and 99.5-percent limits span 205 bins, the estimated occupied bandwidth is

$$B_{99} = 205 \times 488.28125 \approx 100.10 \text{ kHz.} \quad (109)$$

13.3 Instantaneous frequency from a quarter-turn phase step

At $F_s = 2 \text{ MS/s}$, suppose adjacent IQ samples rotate by

$$\Delta\phi = \frac{\pi}{2}. \quad (110)$$

Then

$$f_{\text{inst}} = \frac{2,000,000}{2\pi} \frac{\pi}{2} = 500,000 \text{ Hz.} \quad (111)$$

13.4 Default weighted similarity

Suppose

$$S_M = 0.90, \quad S_F = 0.80, \quad S_B = 0.75. \quad (112)$$

With the default weights,

$$S = 0.30(0.90) + 0.60(0.80) + 0.10(0.75) \quad (113)$$

$$= 0.825. \quad (114)$$

The candidate therefore receives an 82.5 percent composite similarity score.

14 Engineering Notes and Interpretation

1. **Detection is power based, not magnitude based.** This is why dB thresholds use $10^{D/10}$ rather than $10^{D/20}$.
2. **The noise baseline is protected from active bursts.** The exponential baseline is updated while inactive and again after a completed burst, rather than continuously tracking upward through the transmission.
3. **The initial baseline is intentionally low-quantile.** Approximately the 20th percentile reduces contamination by early transmissions.
4. **The OBW value is percentile based.** It is not a threshold-at-3 dB bandwidth. It retains approximately the central 99 percent of accumulated spectral power.
5. **Magnitude and frequency are the only trend arrays currently used in ranking.** Phase trend is extracted, detrended, normalized, cached, and displayed, but the active score does not include a phase-correlation term.
6. **Absolute average frequency is removed.** Frequency trend scoring therefore emphasizes modulation and frequency-shape changes rather than a constant carrier offset.
7. **Linear phase slope is removed.** This suppresses phase ramps associated with constant residual frequency and preserves nonlinear phase-shape changes.

8. **Negative correlation is not treated as similarity.** The NCC routine initializes the best value at zero and clamps the result to $[0, 1]$.
9. **The lag search is bounded.** Default 256-point trends may shift by at most 32 points in either direction, preventing arbitrary alignment from inflating similarity.
10. **Weights are relative scalars.** The denominator normalizes any nonzero user-entered weight sum, so values such as $3/6/1$ are mathematically equivalent to $0.3/0.6/0.1$.

15 Compact Formula Reference

Operation	Equation
Complex16 normalization	$x[n] = I_{16}[n]/32768 + jQ_{16}[n]/32768$
Sample power	$p[n] = x[n] ^2 = I_f[n]^2 + Q_f[n]^2$
Window mean power	$P_w[r] = N_r^{-1} \sum_n x_r[n] ^2$
Power-dB ratio	$R = 10^{D/10}$
Baseline coefficient	$\alpha = e^{-(N_w/F_s)/\tau}$
Adaptive baseline	$B_r = \alpha B_{r-1} + (1 - \alpha)P_w[r]$
Hann window	$w[n] = 0.5 - 0.5 \cos(2\pi n/(N - 1))$
DFT	$X[k] = \sum_n x_w[n] e^{-j2\pi kn/N}$
Spectral power	$S[k] \propto X[k] ^2$
OBW bin width	$\Delta f = F_s/N$
99% OBW	$B_{99} = (k_U - k_L + 1)F_s/N$ at 0.5%/99.5% cumulative limits
Magnitude	$m[n] = \sqrt{I_f[n]^2 + Q_f[n]^2}$
Wrapped phase	$\phi[n] = \text{atan2}(Q_f[n], I_f[n])$
Phase increment	$\Delta\phi[n] = \arg(x[n]x^*[n-1])$
Instantaneous frequency	$f_{\text{inst}}[n] = F_s\Delta\phi[n]/(2\pi)$
Trend bucket	$q = \lfloor rP/N_s \rfloor$
Frequency centering	$F_c[q] = F[q] - \bar{F}$
Phase detrend line	$L[q] = \Phi[0] + (\Phi[P-1] - \Phi[0])q/(P-1)$
Z-score	$z[i] = (v[i] - \mu)/\sigma$
Lagged NCC	$\rho[k] = \sum_i x_i y_{i+k} / \sqrt{\sum_i x_i^2 \sum_i y_{i+k}^2}$
Bandwidth similarity	$S_B = \min(B_q, B_c) / \max(B_q, B_c)$
Query energy proxy	$E = P_{\text{avg}}T$
Composite score	$S = (w_M S_M + w_F S_F + w_B S_B) / (w_M + w_F + w_B)$

16 Core DSP Source Locations

Function / region	Lines	DSP purpose
<code>correlation_parse_recording_metadata</code>	372–436	Converts filename metadata to Hz and samples/s
<code>correlation_normalize_trend</code>	961–1001	Mean, variance, standard deviation, z-score
<code>correlation_detect_ranges</code>	1003–1237	Window power, baseline, hysteresis burst detection
<code>correlation_estimate_occupied_bandwidth</code>	1239–1423	Hann FFT, shifted power accumulation, 99% OBW
<code>correlation_build_signature</code>	1425–1620	Magnitude, frequency, phase, detrending, power features
<code>correlation_extract_file_signatures</code>	1622–1678	Detection-to-signature orchestration
<code>correlation_maximum_normalized_cross_correlation</code>	1680–1771	Bounded-lag shape similarity
<code>correlation_bandwidth_similarity</code>	1773–1789	Symmetric bandwidth ratio
<code>correlation_signature_energy</code>	1791–1804	Query burst energy proxy
<code>correlation_calculate_score</code>	1891–1911	Normalized weighted score
<code>correlation_worker</code>	1931–2171	Query selection and candidate ranking
<code>correlation_draw_trend_line</code>	2720–2762	Normalized signature graph mapping

17 Core DSP Calculation Listings

The following excerpts collect the principal calculation paths in source order. The gutter values are the actual line numbers from the supplied `CorrelationWorkstation.c`.

17.1 Power-window burst detection

CorrelationWorkstation.c lines 1098–1205:

```

1098     for (size_t i = 0; i < wanted; i++) {
1099         double real = (double)samples[2U * i] / 32768.0;
1100         double imag = (double)samples[2U * i + 1U] / 32768.0;
1101         sum += real * real + imag * imag;
1102     }
1103     power[window] = wanted > 0 ? sum / (double)wanted : 0.0;
1104     initial_count = window_count < 500 ? window_count : 500;
1105     initial_values = (double *)malloc(initial_count * sizeof(double));
1106     if (!initial_values) {
1107         free(samples);
1108         free(power);
1109         return 0;
1110     }
1111     memcpy(initial_values, power, initial_count * sizeof(double));
1112     qsort(initial_values, initial_count, sizeof(double), correlation_double_compare);
1113     baseline = initial_values[(initial_count - 1U) / 5U];
1114     free(initial_values);
1115     if (baseline < 1e-15) {
1116         baseline = 1e-15;
1117     }
1118     alpha = exp(-((double>window_samples / sample_rate) / CORRELATION_BASELINE_TIME_CONSTANT_SEC);
1119     low_windows_required = (int)ceil(CORRELATION_END_HOLD_SEC * sample_rate / (double>window_samples);
1120     pretrigger_windows = (int)ceil(CORRELATION_PRETRIGGER_SEC * sample_rate / (double>window_samples);
1121     for (size_t window = 0; window < window_count; window++) {

```

```

1146     double current = power>window];
1148     if (!active) {
1150         if (current > baseline * start_ratio) {
1152             active = 1;
1153             low_window_count = 0;
1154             start_window = window > (size_t)pretrigger_windows ? window - (size_t)pretrigger_windows : 0;
1156         }
1158         else {
1160             baseline = alpha * baseline + (1.0 - alpha) * current;
1162         }
1164     }
1166     else {
1168         if (current < baseline * end_ratio) {
1170             low_window_count++;
1172         }
1174         else {
1176             low_window_count = 0;
1178         }
1180         if (low_window_count >= low_windows_required) {
1182             size_t end_window = window + 1U - (size_t)low_window_count;
1183             size_t start_sample = start_window * window_samples;
1184             size_t end_sample = end_window * window_samples;
1186             if (end_sample > iq_count) {
1188                 end_sample = iq_count;
1190             }
1192             if (end_sample > start_sample &&
1193                 (double)(end_sample - start_sample) / sample_rate >= CORRELATION_MIN_BURST_SEC &&
1194                 range_count < CORRELATION_MAX_SIGNATURES_PER_FILE) {
1196                 ranges[range_count].start_sample = start_sample;
1197                 ranges[range_count].end_sample = end_sample;
1198                 range_count++;
1200             }
1202             active = 0;
1203             low_window_count = 0;
1204             baseline = alpha * baseline + (1.0 - alpha) * current;

```

17.2 Occupied-bandwidth FFT calculation

CorrelationWorkstation.c lines 1321–1395:

```

1321     for (int i = 0; i < CORRELATION_OBW_FFT_SIZE; i++) {
1322         double window = 0.0;
1323         double real = 0.0;
1324         double imag = 0.0;
1326         if ((size_t)i < wanted) {
1328             window = 0.5 - 0.5 * cos((2.0 * M_PI * (double)i) / (double)(CORRELATION_OBW_FFT_SIZE - 1));
1329             real = (double)samples[2U * (size_t)i] / 32768.0;
1330             imag = (double)samples[2U * (size_t)i + 1U] / 32768.0;
1332         }
1334         input[i][0] = real * window;
1335         input[i][1] = imag * window;
1336     }
1338     fftw_execute(plan);
1340     for (int i = 0; i < CORRELATION_OBW_FFT_SIZE; i++) {
1341         int shifted = (i + CORRELATION_OBW_FFT_SIZE / 2) % CORRELATION_OBW_FFT_SIZE;
1342         double real = output[i][0];
1343         double imag = output[i][1];
1344         spectrum[shifted] += real * real + imag * imag;
1345     }
1347     block_count++;
1348     correlation_worker_checkpoint(CORRELATION_HIDDEN_YIELD_MS);
1350     if (segment_samples <= CORRELATION_OBW_FFT_SIZE) {
1352         break;
1354     }
1355 }
1357 if (block_count > 0) {
1359     double total = 0.0;
1360     double lower_target;

```

```

1361     double upper_target;
1362     double cumulative = 0.0;
1363     int lower_bin = 0;
1364     int upper_bin = CORRELATION_OBW_FFT_SIZE - 1;
1365     int lower_found = 0;
1366     for (int i = 0; i < CORRELATION_OBW_FFT_SIZE; i++) {
1367         total += spectrum[i];
1368     }
1369     lower_target = total * 0.005;
1370     upper_target = total * 0.995;
1371     if (total > DBL_MIN) {
1372         for (int i = 0; i < CORRELATION_OBW_FFT_SIZE; i++) {
1373             cumulative += spectrum[i];
1374             if (!lower_found && cumulative >= lower_target) {
1375                 lower_bin = i;
1376                 lower_found = 1;
1377             }
1378             if (cumulative >= upper_target) {
1379                 upper_bin = i;
1380                 break;
1381             }
1382         }
1383     }
1384     bandwidth = ((double)(upper_bin - lower_bin + 1) / (double)CORRELATION_OBW_FFT_SIZE) *
1385     sample_rate;

```

17.3 Magnitude, instantaneous-frequency, and unwrapped-phase calculation

CorrelationWorkstation.c lines 1500–1562:

```

1500     for (size_t i = 0; i < wanted; i++) {
1501         size_t relative_index = processed + i;
1502         int point = (int)((relative_index * (size_t)Global_Correlation_Trend_Points) / segment_samples);
1503         double real = (double)samples[2U * i] / 32768.0;
1504         double imag = (double)samples[2U * i + 1U] / 32768.0;
1505         double magnitude = hypot(real, imag);
1506         double power = real * real + imag * imag;
1507         if (point >= Global_Correlation_Trend_Points) {
1508             point = Global_Correlation_Trend_Points - 1;
1509         }
1510         magnitude_sum[point] += magnitude;
1511         magnitude_count[point]++;
1512         total_power += power;
1513         if (power > peak_power) {
1514             peak_power = power;
1515         }
1516         {
1517             double phase = atan2(imag, real);
1518             if (have_previous) {
1519                 double product_real = real * previous_real + imag * previous_imag;
1520                 double product_imag = imag * previous_real - real * previous_imag;
1521                 double instantaneous_frequency = (sample_rate / (2.0 * M_PI)) * atan2(product_imag,
1522                 product_real);
1523                 double phase_delta = phase - previous_phase;
1524                 while (phase_delta > M_PI) {
1525                     phase_delta -= 2.0 * M_PI;
1526                 }
1527                 while (phase_delta < -M_PI) {
1528                     phase_delta += 2.0 * M_PI;
1529                 }
1530                 unwrapped_phase += phase_delta;
1531                 frequency_sum[point] += instantaneous_frequency;
1532                 frequency_count[point]++;
1533             }
1534             else {
1535                 unwrapped_phase = phase;
1536             }
1537             phase_sum[point] += unwrapped_phase;
1538             phase_count[point]++;

```

```

1556         previous_phase = phase;
1557     }
1559     previous_real = real;
1560     previous_imag = imag;
1561     have_previous = 1;
1562 }

```

17.4 Trend centering, detrending, and normalization

CorrelationWorkstation.c lines 1576–1617:

```

1576     for (int point = 0; point < Global_Correlation_Trend_Points; point++) {
1577         signature->magnitude_trend[point] =
1578             magnitude_count[point] > 0 ? (float)(magnitude_sum[point] / (double)magnitude_count[point]) :
0.0f;
1579         signature->frequency_trend[point] =
1580             frequency_count[point] > 0 ? (float)(frequency_sum[point] / (double)frequency_count[point]) :
0.0f;
1581         signature->phase_trend[point] =
1582             phase_count[point] > 0 ? (float)(phase_sum[point] / (double)phase_count[point]) : 0.0f;
1584         if (frequency_count[point] > 0) {
1586             frequency_mean += signature->frequency_trend[point];
1587             frequency_points++;
1589         }
1590     }
1592     if (frequency_points > 0) {
1594         frequency_mean /= (double)frequency_points;
1596     }
1598     for (int point = 0; point < Global_Correlation_Trend_Points; point++) {
1599         double phase_position =
1600             Global_Correlation_Trend_Points > 1 ? (double)point / (double)(Global_Correlation_Trend_Points -
1) : 0.0;
1601         double phase_line =
1602             (double)signature->phase_trend[0] +
1603             ((double)signature->phase_trend[Global_Correlation_Trend_Points - 1] -
(double)signature->phase_trend[0]) *
1604             phase_position;
1606         signature->frequency_trend[point] = (float)((double)signature->frequency_trend[point] -
frequency_mean);
1607         /* Remove constant carrier rotation so the graph shows phase-shape changes. */
1608         signature->phase_trend[point] = (float)((double)signature->phase_trend[point] - phase_line);
1609     }
1611     correlation_normalize_trend(signature->magnitude_trend, Global_Correlation_Trend_Points);
1612     correlation_normalize_trend(signature->frequency_trend, Global_Correlation_Trend_Points);
1613     correlation_normalize_trend(signature->phase_trend, Global_Correlation_Trend_Points);
1614     signature->peak_power = peak_power;
1615     signature->average_power = segment_samples > 0 ? total_power / (double)segment_samples : 0.0;
1616     signature->occupied_bandwidth =
1617         correlation_estimate_occupied_bandwidth(fp, range.start_sample, range.end_sample, sample_rate);

```

17.5 Bounded-lag normalized cross-correlation

CorrelationWorkstation.c lines 1687–1770:

```

1687     double best = 0.0;
1688     double full_left_energy = 0.0;
1689     double full_right_energy = 0.0;
1690     int point_count = Global_Correlation_Trend_Points;
1691     int maximum_lag;
1693     if (point_count < CORRELATION_MIN_TREND_POINTS) {
1695         point_count = CORRELATION_MIN_TREND_POINTS;
1697     }
1699     if (point_count > CORRELATION_MAX_TREND_POINTS) {
1701         point_count = CORRELATION_MAX_TREND_POINTS;
1703     }
1705     maximum_lag = (int)((double)point_count * CORRELATION_MAX_LAG_PERCENT);

```

```

1707     if (maximum_lag < 1) {
1709         maximum_lag = 1;
1711     }
1713     for (int i = 0; i < point_count; i++) {
1714         full_left_energy += (double)left[i] * (double)left[i];
1715         full_right_energy += (double)right[i] * (double)right[i];
1716     }
1718     if (full_left_energy <= DBL_MIN && full_right_energy <= DBL_MIN) {
1720         return 1.0;
1722     }
1724     if (full_left_energy <= DBL_MIN || full_right_energy <= DBL_MIN) {
1726         return 0.0;
1728     }
1730     for (int lag = -maximum_lag; lag <= maximum_lag; lag++) {
1731         double numerator = 0.0;
1732         double left_energy = 0.0;
1733         double right_energy = 0.0;
1734         int count = 0;
1736         for (int i = 0; i < point_count; i++) {
1737             int j = i + lag;
1739             if (j < 0 || j >= point_count) {
1741                 continue;
1743             }
1745             numerator += (double)left[i] * (double)right[j];
1746             left_energy += (double)left[i] * (double)left[i];
1747             right_energy += (double)right[j] * (double)right[j];
1748             count++;
1749         }
1751         if (count >= point_count / 2 && left_energy > DBL_MIN && right_energy > DBL_MIN) {
1753             double score = numerator / sqrt(left_energy * right_energy);
1755             if (score > best) {
1757                 best = score;
1759             }
1761         }
1763         if (((unsigned int)(lag + maximum_lag + 1) % CORRELATION_HIDDEN_NCC_LAG_BATCH) == 0) {
1765             correlation_worker_checkpoint(CORRELATION_HIDDEN_YIELD_MS);
1767         }
1768     }
1770     return correlation_clamp_double(best, 0.0, 1.0);

```

17.6 Similarity components and composite score

CorrelationWorkstation.c lines 1773–1804 and 1891–1910:

```

1773 static double correlation_bandwidth_similarity(double left, double right) {
1774     /*
1775      * Purpose: Compares occupied bandwidth values
1776      * Returns: Similarity from zero to one
1777      */
1779     double maximum = left > right ? left : right;
1780     double minimum = left < right ? left : right;
1782     if (maximum <= 1.0) {
1784         return maximum == minimum ? 1.0 : 0.0;
1786     }
1788     return correlation_clamp_double(minimum / maximum, 0.0, 1.0);
1789 }
1791 static double correlation_signature_energy(const TransmissionSignature *signature) {
1792     /*
1793      * Purpose: Ranks candidate query bursts by approximate captured energy
1794      * Returns: Energy score
1795      */
1797     if (!signature) {
1799         return 0.0;
1801     }
1803     return signature->average_power * signature->duration;
1804 }
1891 static double correlation_calculate_score(double magnitude_score, double frequency_score, double
    bandwidth_score) {

```

```

1892  /*
1893      Purpose: Combines component similarities using user-entered score scalars
1894      Returns: Normalized score from zero to one
1895  */
1896  double weight_sum =
1897      Global_Correlation_Magnitude_Weight + Global_Correlation_Frequency_Weight +
1898      Global_Correlation_Bandwidth_Weight;
1900  if (weight_sum <= DBL_MIN) {
1902      return 0.0;
1904  }
1906  return correlation_clamp_double((Global_Correlation_Magnitude_Weight * magnitude_score +
1907      Global_Correlation_Frequency_Weight * frequency_score +
1908      Global_Correlation_Bandwidth_Weight * bandwidth_score) /
1909      weight_sum,
1910      0.0, 1.0);

```

17.7 Active candidate-ranking calculation

CorrelationWorkstation.c lines 1990–2002 and 2051–2074:

```

1990  for (uint32_t i = 0; i < Global_Correlation_Cache[query_cache_index].signature_count; i++) {
1991      double energy =
1992      correlation_signature_energy(&Global_Correlation_Cache[query_cache_index].signatures[i]);
1993      if (energy > best_query_energy) {
1995          best_query_energy = energy;
1996          best_query_signature = (int)i;
1998      }
1999  }
2001  Global_Correlation_Query_Signature =
2002  Global_Correlation_Cache[query_cache_index].signatures[best_query_signature];
2003  Global_Correlation_Query_Signature_Valid = 1;
2051  for (uint32_t signature_index = 0; signature_index <
2052  Global_Correlation_Cache[cache_index].signature_count;
2053  signature_index++) {
2054      TransmissionSignature *stored =
2055      &Global_Correlation_Cache[cache_index].signatures[signature_index];
2056      double magnitude_score = correlation_maximum_normalized_cross_correlation(
2057      Global_Correlation_Query_Signature.magnitude_trend, stored->magnitude_trend);
2058      double frequency_score = correlation_maximum_normalized_cross_correlation(
2059      Global_Correlation_Query_Signature.frequency_trend, stored->frequency_trend);
2060      double bandwidth_score = correlation_bandwidth_similarity(
2061      Global_Correlation_Query_Signature.occupied_bandwidth, stored->occupied_bandwidth);
2062      double score = correlation_calculate_score(magnitude_score, frequency_score, bandwidth_score);
2063      correlation_worker_checkpoint(CORRELATION_HIDDEN_YIELD_MS);
2064      if (!have_result || score > best_result.score) {
2066          best_result.signature_index = (int)signature_index;
2067          best_result.score = score;
2068          best_result.magnitude_score = magnitude_score;
2069          best_result.frequency_score = frequency_score;
2070          best_result.bandwidth_score = bandwidth_score;
2071          best_result.signature = *stored;
2072          have_result = 1;
2074      }

```